

Section 6 — Chat Results (RAG Output) Explained

This document explains Section 6 of ResearchLens: the AI chat interface that answers questions about your uploaded research papers. It describes the UI, the live question support, and how RAG works inside this project.

1. What is Section 6?

Section 6 is the ResearchLens chatbot interface. The goal is to let you ask natural language questions about your dataset and get answers grounded in the uploaded papers.

What you see in the UI

- **Live question support** means the chat responds to your queries in real time.
- **Ask anything about your dataset** is a prompt to encourage dataset-specific queries.
- **Your question** is the input field where you type the query.
- **Feel free to ask queries...** is a placeholder hint.
- **Ask question** is the button to submit.
- **Press Enter to send, Shift+Enter for new line.** is the typing behavior.
- **Answer** is the area where the chatbot reply appears.
- **How RAG works:** explains the retrieval-augmented generation concept.

2. How the input works

The chat UI is implemented in `frontend/src/pages/dashboard/sections/ChatbotSection.tsx`.

Typing and submission

- Press **Enter** to send your question.
- Press **Shift+Enter** to insert a new line in the input box.
- The user message is appended to the chat history immediately.
- The interface then sends the question to the backend if papers are loaded.

Live support behavior

If the app has papers loaded, the chat will use the real backend RAG pipeline. If not, it uses a mock fallback response generator to simulate answers. That means the UI still works without data, but the answers are not real unless actual papers are available.

3. What RAG means here

RAG stands for Retrieval-Augmented Generation. In ResearchLens, it means:

1. split the dataset into small text chunks,
2. convert user question and chunks into vectors,
3. find the most relevant chunks for the question,
4. build a prompt with those chunks plus topic/gap/trend summaries,
5. send the prompt to the local Ollama backend,

6. return the generated answer with citations.

4. Backend logic for RAG

The backend implementation lives in `backend/src/services/module6Chatbot.js`. This module handles the actual retrieval and generation steps.

Step 1: Create chunks

The code reads each paper's abstract and content and splits it into chunks of about 3 sentences. Each chunk contains:

- `chunkId`
- `paperId`
- `title`
- `text`

This turns the dataset into a set of retrievable passages.

Step 2: Convert text to vectors

The module tokenizes each chunk and the user question. It builds a shared vocabulary and vectorizes both question and chunks using that vocabulary. Then it computes cosine similarity to rank chunks by relevance to the question.

Step 3: Rank and select top chunks

Chunks are sorted by similarity to the question. The top 6 chunks are kept for the prompt. This is the retrieval stage.

Step 4: Build the Ollama prompt

`buildOllamaPrompt()` assembles the context sent to the Ollama model. It includes:

- a short corpus summary,
- titles of the top papers,
- top detected topics,
- top research gaps,
- top trends,
- the most relevant text chunks,
- the user question,
- instructions to answer from the corpus and cite papers.

This ensures the generated answer is grounded in the local dataset.

Step 5: Call Ollama

The backend sends the prompt to the local Ollama API at `http://127.0.0.1:11434/v1/completions`. The model name is taken from `OLLAMA_MODEL`, defaulting to `llama3.1:8b`. The response text is returned as the chatbot answer.

Step 6: Create citations

The backend also returns citation information from the top-ranked chunks. Each citation includes:

- `paperId`
- `title`
- `chunkId`
- `relevance`

Only chunks with similarity > 0.05 are included. These are shown below the answer in the chat UI.

5. Frontend request flow

The frontend uses the `askChatbot()` API helper in `frontend/src/lib/api.ts`. This sends a POST request to `/api/modules/6-chatbot` with:

- `papers` — the current uploaded papers,
- `question` — the user query.

If the backend call succeeds, the returned answer and citations are displayed. If it fails, the chat shows an error message asking to check the backend.

6. What happens without a real backend?

If `papers.length === 0`, the frontend uses a mock fallback generator instead of the real backend. This fallback lives inside `ChatbotSection.tsx` and returns canned answers based on keyword matching. In real usage, you want actual papers loaded so the real RAG pipeline is used.

7. What the answer looks like

The chatbot message shows:

- the generated text answer,
- a citation list under the answer,
- each citation has a relevance bar and percentage.

The UI formats Markdown-style bold text from `**bold**` markers.

8. Example query flow

Example 1: Ask about gaps

User asks: `What are the top research gaps in this dataset?`

Flow:

- backend ranks relevant chunks,
- prompt includes gap summaries,
- Ollama returns a focused answer,
- citations list the source papers.

Example 2: Ask about topics

User asks: **What topics are present in my dataset?**

Flow:

- backend ranks chunks and includes topic names,
- answer explains the detected topics,
- citations reference relevant papers.

Example 3: Ask about trends

User asks: **Which topics are rising?**

Flow:

- backend includes trend summaries,
- answer describes rising topics,
- citations point to papers in those topics.

9. Why this matters

This section is useful because it turns your uploaded dataset into an interactive research assistant. Instead of reading raw papers, you can ask natural questions and get answers grounded in the same papers.

That is the promise of RAG: the model does not hallucinate from generic knowledge alone. It is given a local corpus and asked to answer from it.

10. Limitations to be aware of

- The current retrieval uses simple keyword-vector cosine similarity, not a full embedding model.
- The local Ollama backend must be running for real answers.
- If the backend is unavailable, the UI falls back to mock/demo responses.
- Citation precision is approximate and based on chunk relevance scores.

File saved

This explanation is saved in **research-chat-explanation.md** in the project root.